# Aula 7 – Árvores AVL

Inserção

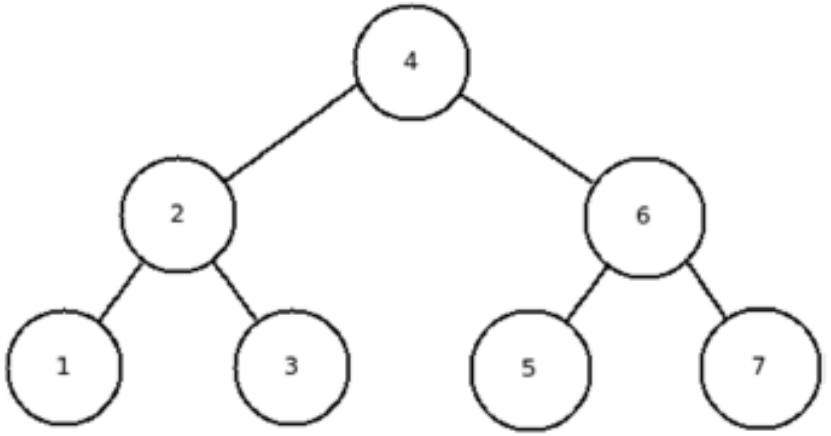
Pesquisa

Eliminação

Exemplos

# Árvores **A**delson-**V**elskii e **L**andis

Uma árvore AVL é uma árvore de pesquisa binária cujos algoritmos de inserção e remoção fazem a árvore ficar sempre balanceada.




A diferença entre a sub-árvore esquerda e a sub-árvore direita é no máximo de **um** nível.

# Árvores AVL

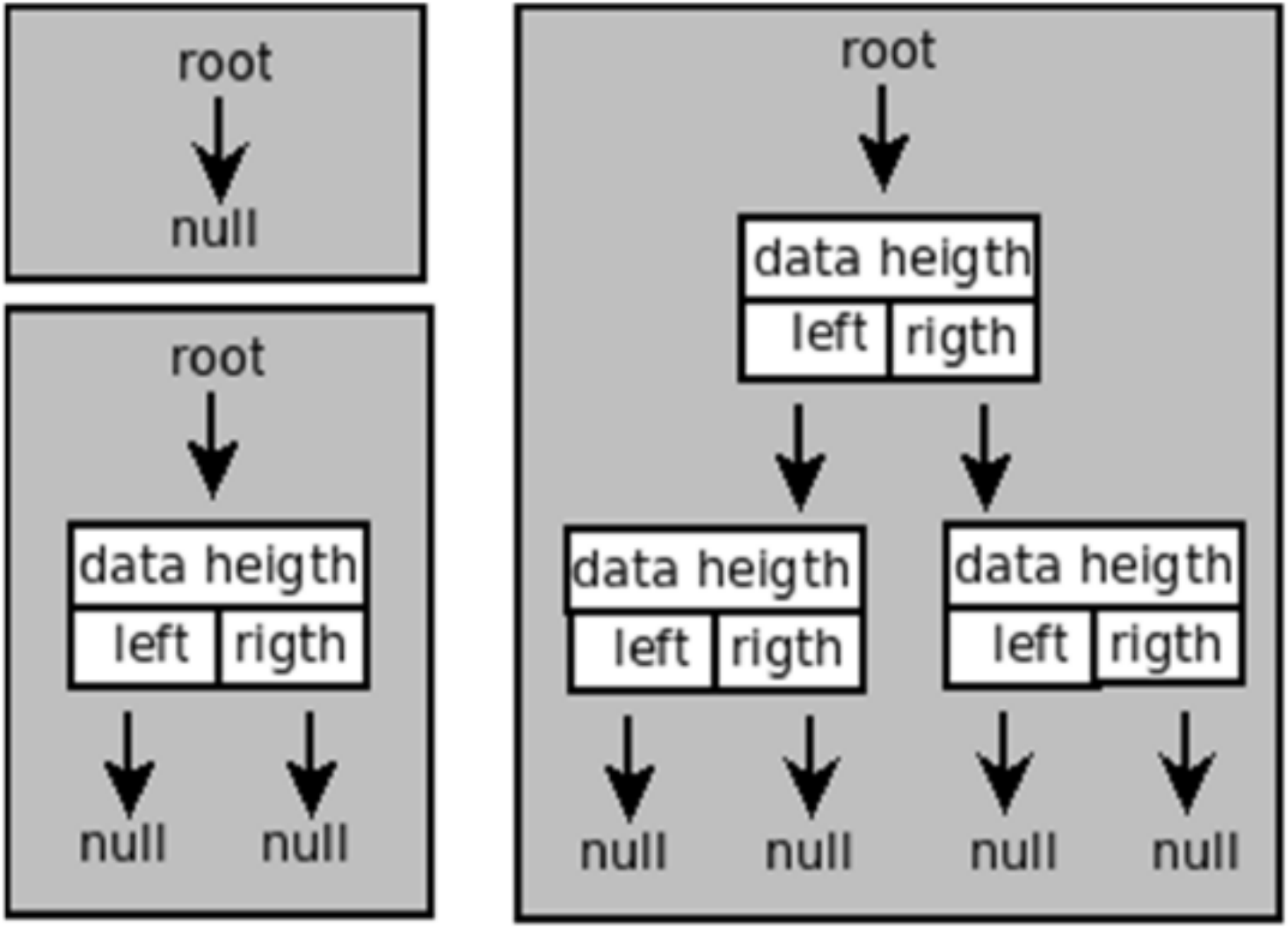
Inserção O(logn) - Remoção O(logn) - Pesquisa O(logn)

Métodos:

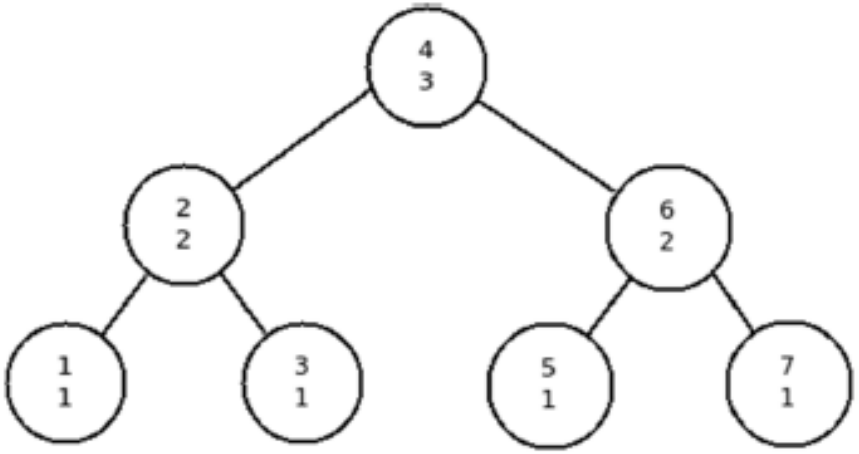
- void add(Object o); // adiciona um objecto na árvore
- boolean contains(Object o); // se contem um objecto
- boolean remove(Object o); // remove um objecto
- Object findMin(); // Devolve o menor objecto da árvore
- Object findMax(); // Devolve o maior objecto da árvore

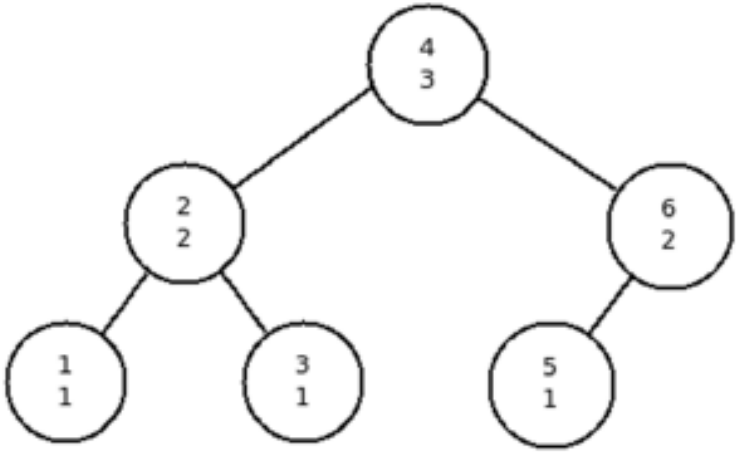
# AVL - Representação

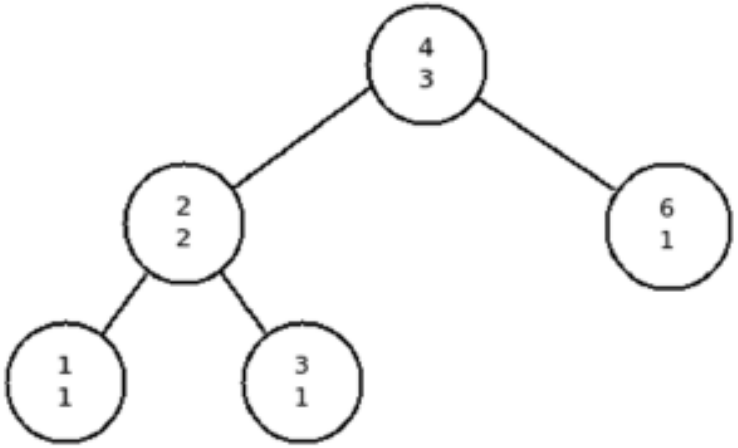
```
Nodo {
    Object data
    int heigth*
    Nodo left
    Nodo rigth
}
Tree {
    Nodo root
}
* Altura do nodo
```

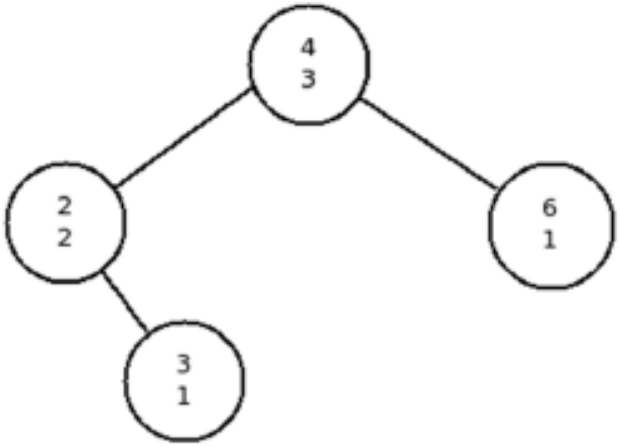


# AVL – Altura de um Nodo



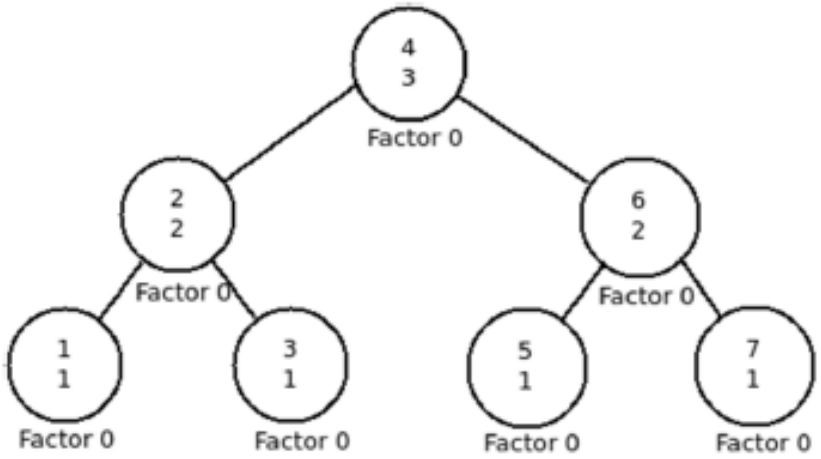



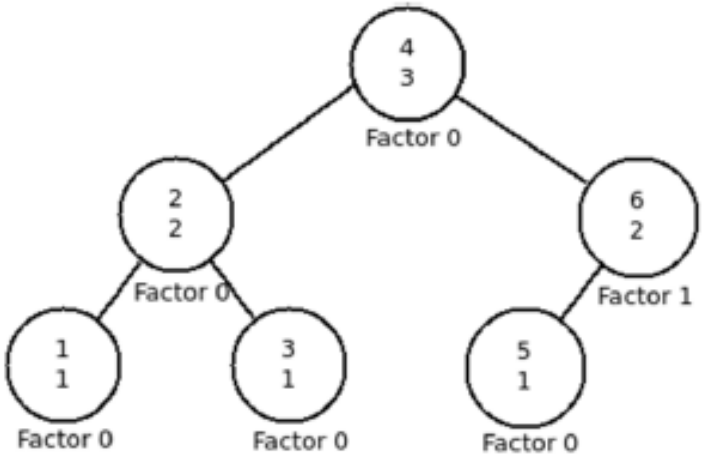



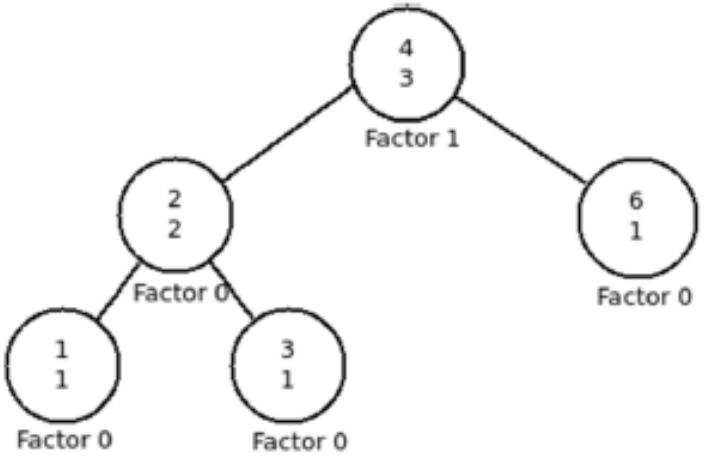



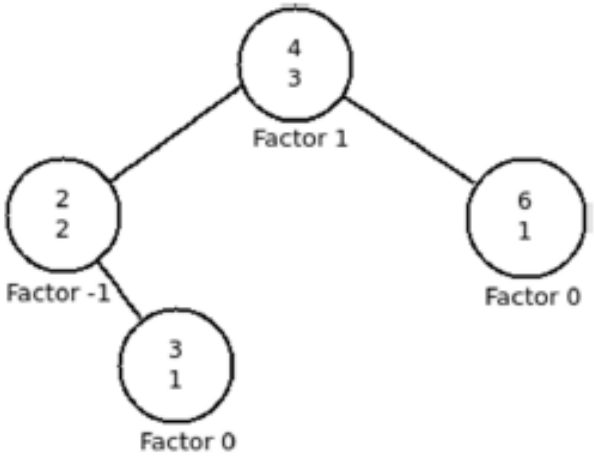
# AVL – Factor de um Nodo

Diferença entre a altura do nodo esquerdo pela altura do nodo direito. Se o nodo não existir a altura é 0.




(c) Paulo Santos

# AVL – Factor de um Nodo

Se o Factor é 0, o nodo está equilibrado.

Se o Factor é 1, o nodo está pesado à esquerda

Se o Factor é -1, o nodo está pesado à direita

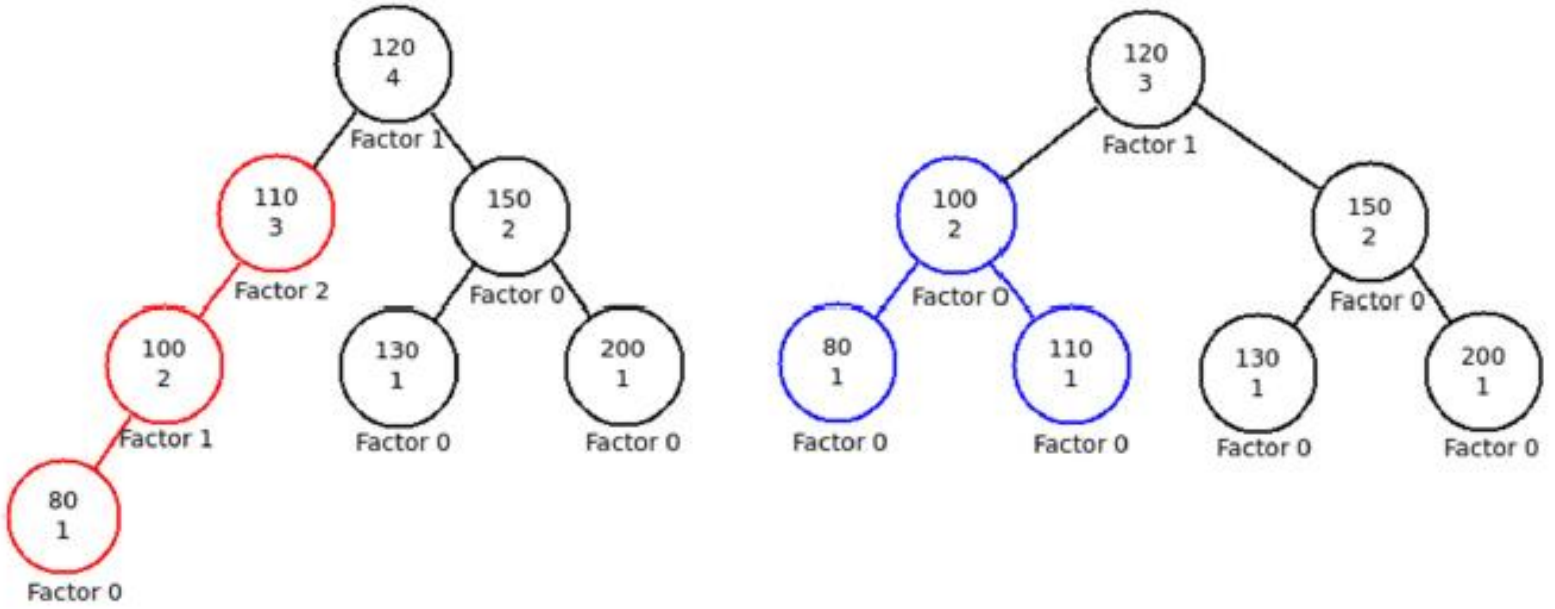
Se o Factor for qualquer outro valor é porque a árvore não está balanceada

```
int factor(Node nodo) {
        return heigth(nodo.left) – heigth(nodo.rigth);
}
```

**O método heigth devolve 0 se o objecto passado por parâmetro for nulo, caso contrario devolve o atributo heigth do nodo.**

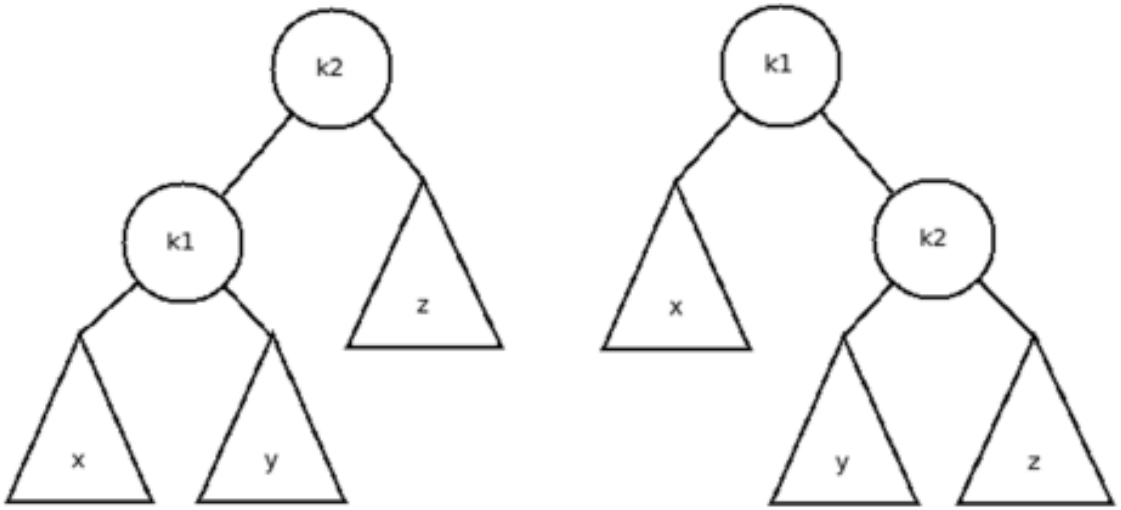
# AVL – Rotação simples à direita

Sempre que um nodo fica com um factor acima de 1 e o seu nodo esquerdo tem um factor maior ou igual a 0 efectua-se uma rotação simples à direita



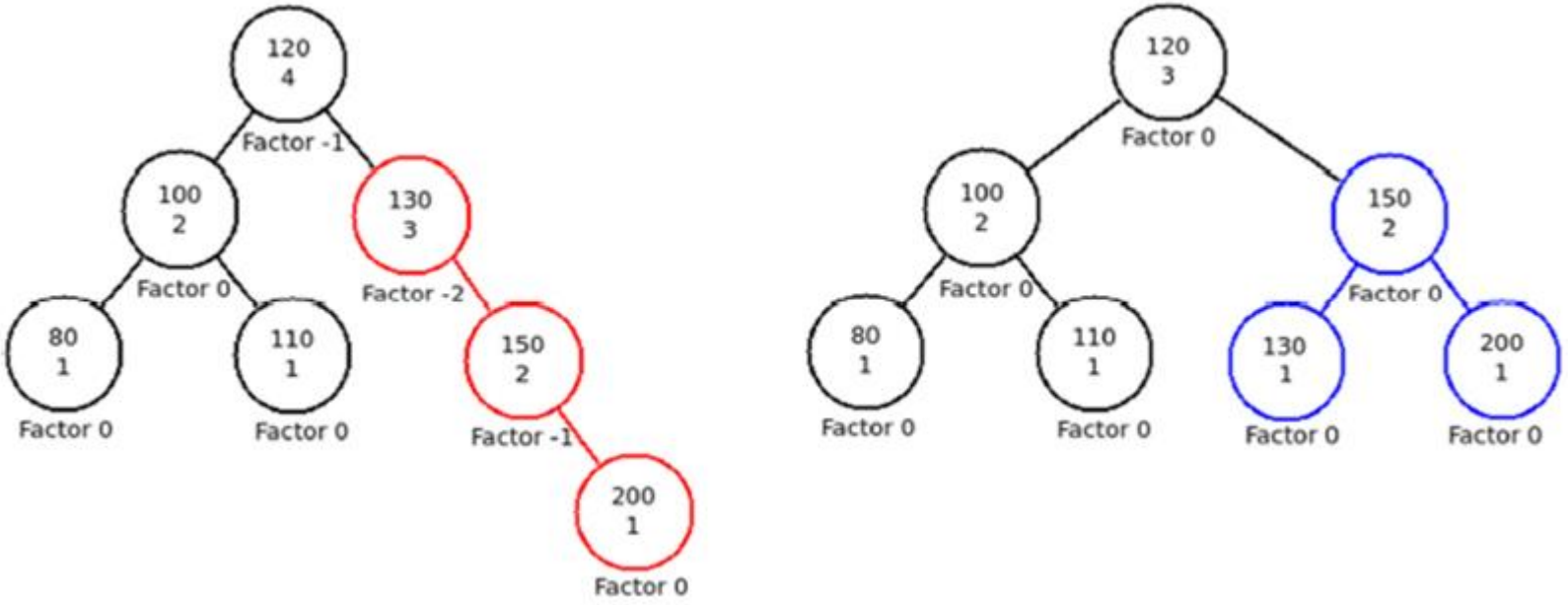
# AVL – Rotação simples à direita

```
Nodo rigthRotation(Nodo k2) {
    k1 = k2.left
    k2.left = k1.rigth
    k1.rigth = k2
    k2.heigth = maximo ( heigth(k2.left), heigth(k2.rigth) ) + 1
    k1.heigth = maximo ( heigth(k1.left), heigth(k2) ) + 1
    devolve k1
}
```



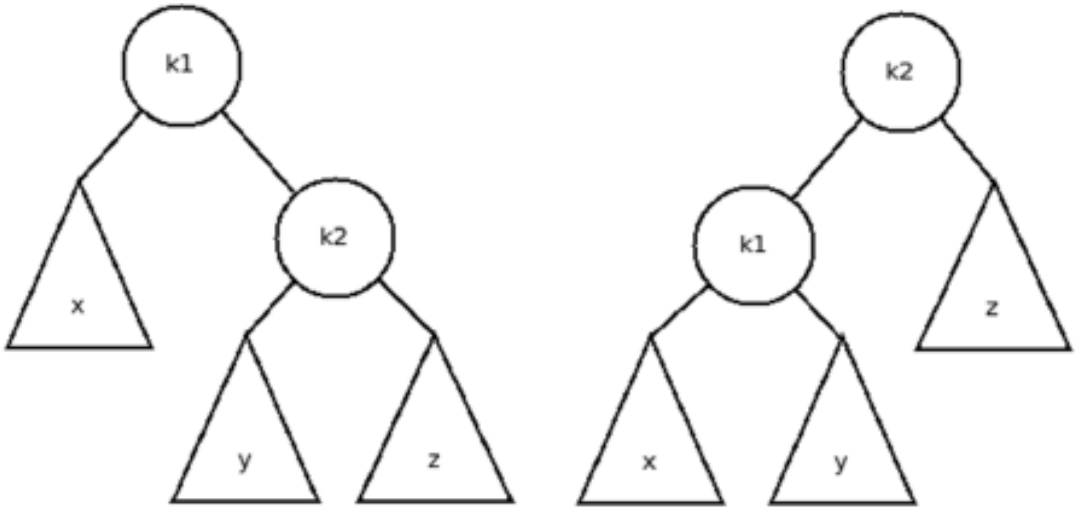
# AVL – Rotação simples à esquerda

Sempre que um nodo fica com um factor abaixo de -1 e o seu nodo direito tem um factor menor ou igual a 0 efectua-se uma rotação simples à esquerda



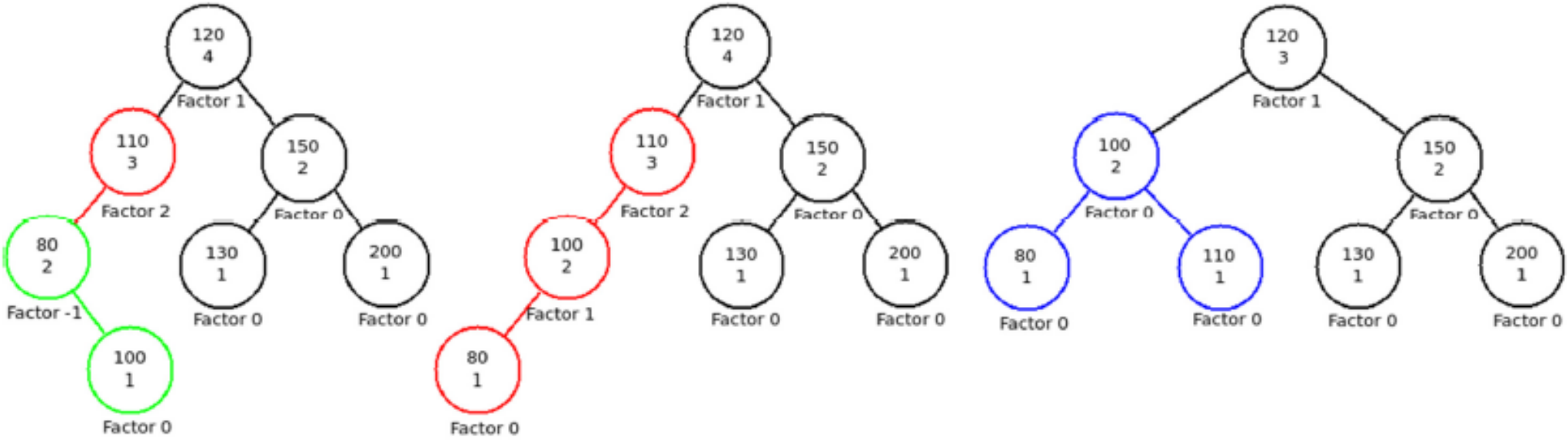
# AVL – Rotação simples à esquerda

```
Nodo leftRotation(Nodo k1) {
    k2 = k1.rigth
    k1.rigth = k2.left
    k2.left = k1
    k1.heigth = maximo ( heigth(k1.left), heigth(k1.rigth) ) + 1
    k2.heigth = maximo ( heigth(k2.rigth), heigth(k1) ) + 1
    devolve k2
}
```



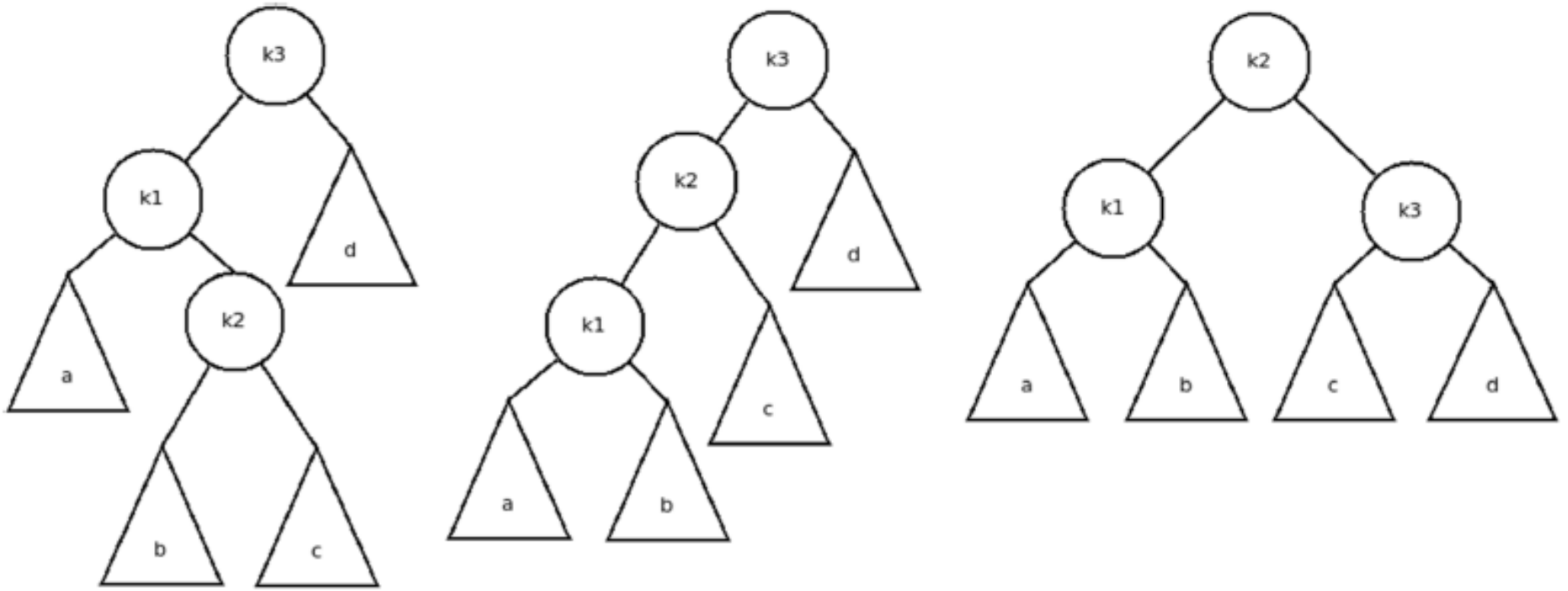
# AVL – Rotação dupla à direita

Sempre que um nodo fica com um factor acima de 1 e o seu nodo esquerdo tem um factor menor que 0 efectua-se uma rotação dupla à direita



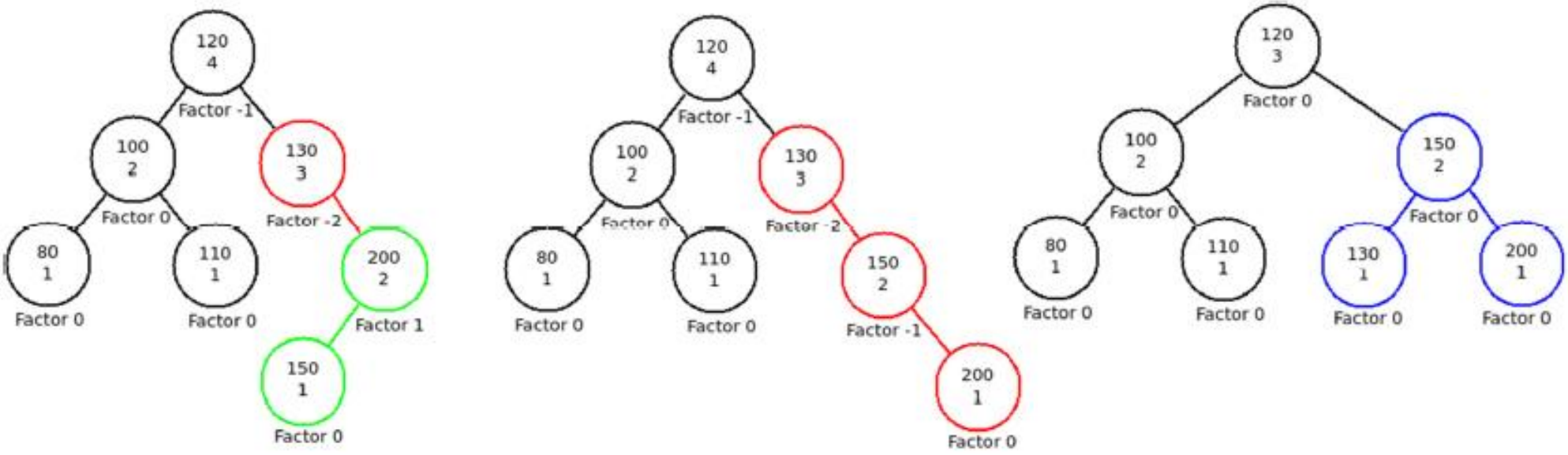
# AVL – Rotação dupla à direita

```
Nodo doubleRigthRotation(Nodo k3) {
    k3.left = leftRotation(k3.left)
    devolve rigthRotation(k3)
}
```



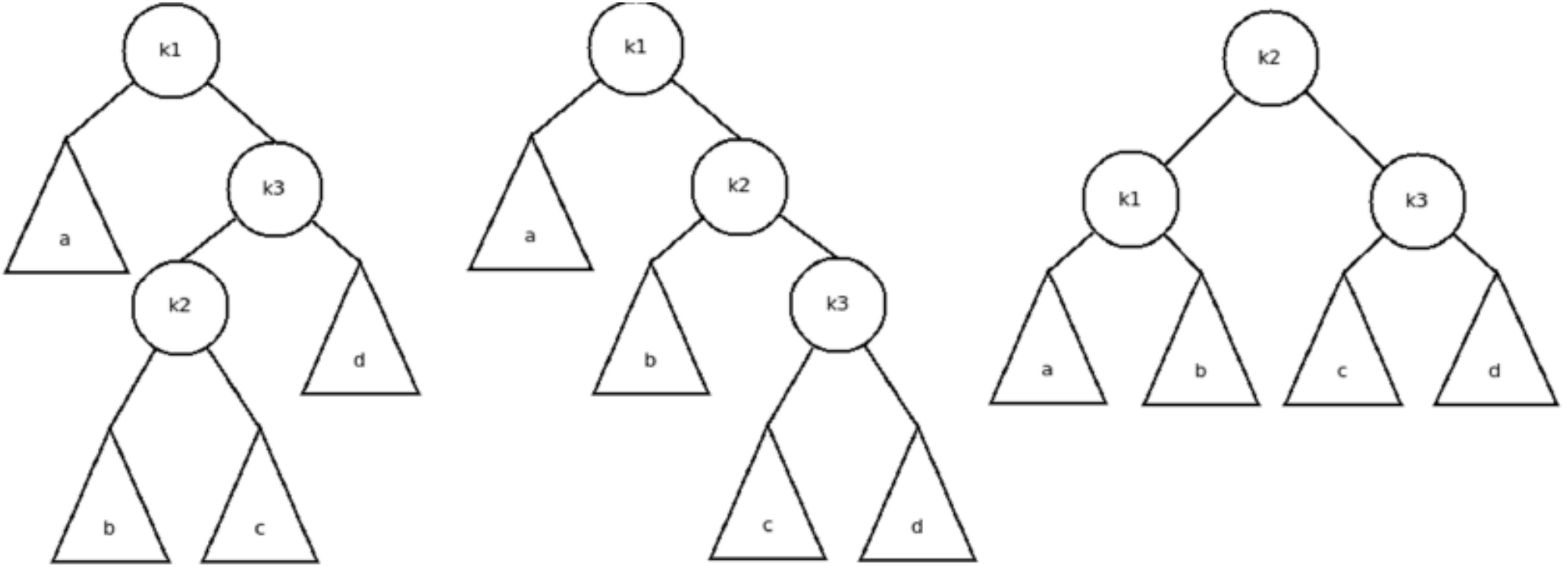
# AVL – Rotação dupla à esquerda

Sempre que um nodo fica com um factor abaixo de -1 e o seu nodo direito tem um factor maior que 1 efectua-se uma rotação dupla à esquerda



# AVL – Rotação dupla à esquerda

```
Nodo doubleLeftRotation(Nodo k1) {
    k1.rigth = rigthRotation(k1.rigth)
    devolve leftRotation(k1)
}
```



# AVL - Inserção

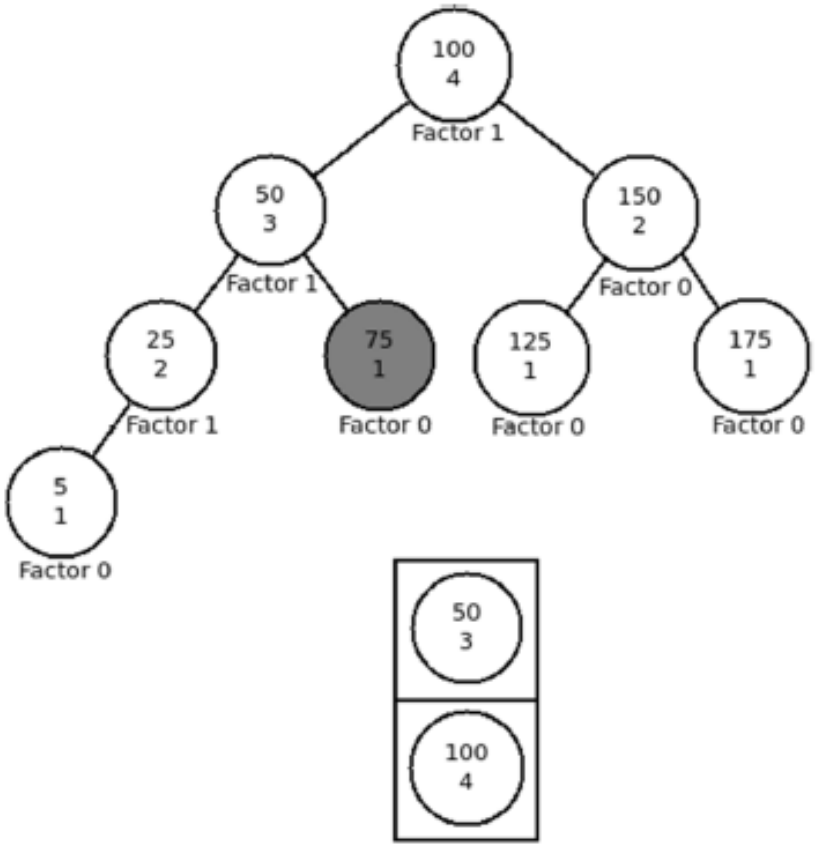
```
void add(Object o) {
    novo(nodo)
    nodo.data = o
    nodo.heigth = 1
    nodo.left = null
    nodo.rigth = null
    Se root = null
        root = nodo
    Se não
        add(root, nodo)
}
```

```
void add(Nodo actual, Nodo novo)
    {
    Se novo.data < actual.data
        Se actual.left = null
            actual.left = novo
        Se não
            add(actual.left, novo)
    Se não
        Se actual.rigth = null
            actual.rigth = novo
        Se não
            add(actual.rigth, novo)
    balance(actual)
}
```

# AVL - Balance

```
void balance(Node nodo) {
    nodo.heigth = maximo( heigth(nodo.left), heigth(nodo.rigth) ) + 1
    Se factor(nodo) > 1
        Se factor(nodo.left) >= 0
            nodo = rigthRotation(nodo)
        Senão
            nodo = doubleRigthRotation(nodo)
    Senão Se factor(nodo) < -1
        Se factor(nodo.rigth) <= 0
            nodo = leftRotation(nodo)
        Senão
            nodo = doubleLeftRotation(nodo)
}
```

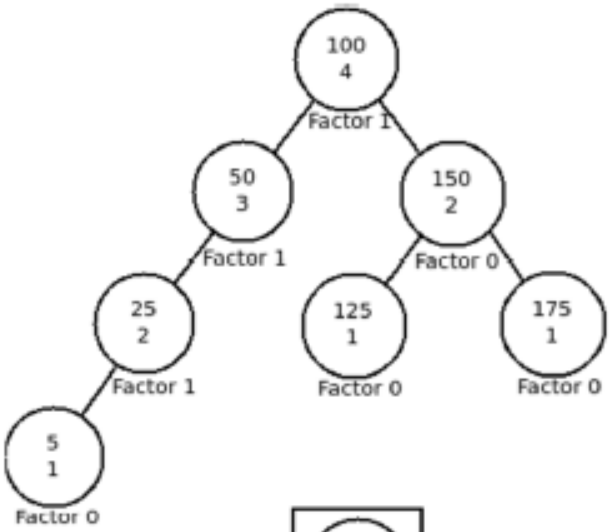
# AVL - Remove

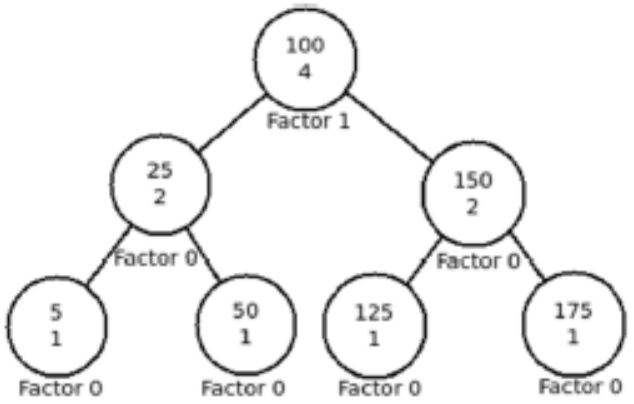


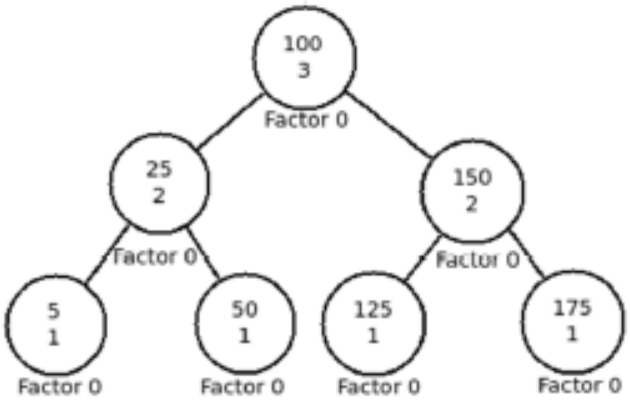

Guarda-se o caminho numa pilha até chegar ao nodo 75.

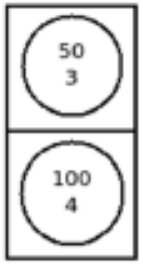
No final da remoção, balanceia-se todos os elementos da pilha

# AVL - Remove


100
4
Factor 1
50
3
Factor 1
150
2
Factor 0
25
2
Factor 1
125
1
Factor 0
175
1
Factor 0
5
1
Factor 0


100
4
Factor 1
25
2
Factor 0
150
2
Factor 0
5
1
Factor 0
50
1
Factor 0
125
1
Factor 0
175
1
Factor 0

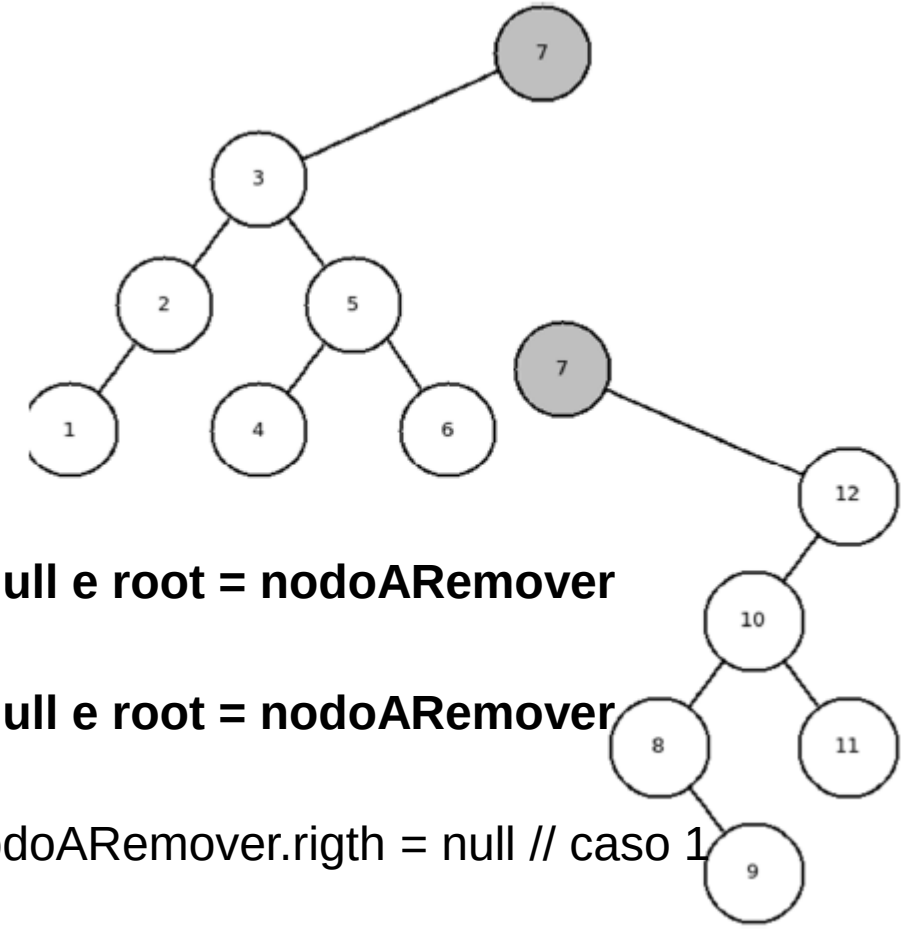

100
3
Factor 0
25
2
Factor 0
150
2
Factor 0
5
1
Factor 0
50
1
Factor 0
125
1
Factor 0
175
1
Factor 0


50
3
100
4


100
4

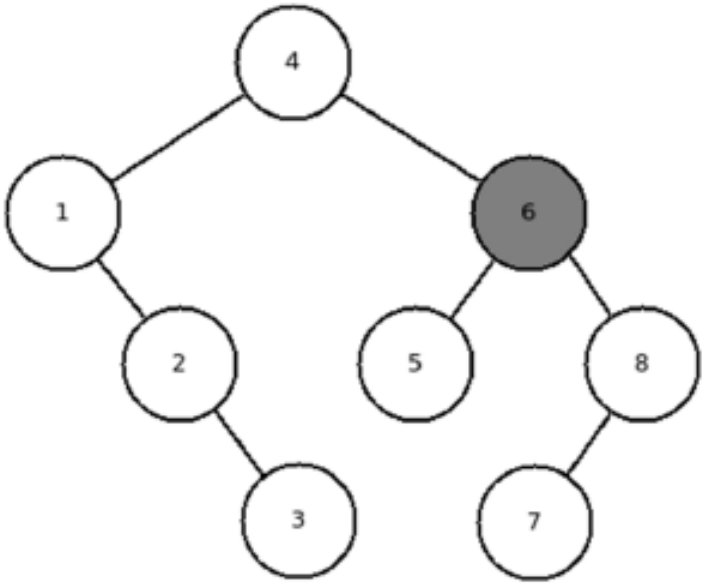
# APB – Remoção

```
boolean remove(Node actual, Object o) {
    nodoARemover = find(o)
    Se nodoARemover = null
        Devolve falso
    Senão
        parent = findParent(o);
        Se root.left = null e root.rigth = null
            root = null
        Senão  root.left = null e root.rigth <> null e root = nodoARemover
            root = root.rigth
        Senão  root.left <> null e root.rigth = null e root = nodoARemover
            root = root.left
        Senão Se nodoARemover.left = null e nodoARemover.rigth = null // caso 1
            Se nodoARemove.data < parent.data
                parent.left = null
            Senão
                parent.rigth = null
```



# APB – Remoção

```
    Senão Se nodoARemover.left = null e nodoARemover.rigth <> null // caso 2.1
        Se nodoARemover.data < parent.data
            parent.left = nodoARemover.rigth
        Senão
            parent.rigth = nodoARemover.rigth
    Senão Se nodoARemover.left <> null e nodoARemover.rigth = null // caso 2.2
        Se nodoARemover.data < parent.data
            parent.left = nodoARemover.left
        Senão
            parent.rigth = nodoARemover.left
     Senão // caso 3
            nodoMaior = nodoARemover.left
            Enquanto nodoMaior.rigth <> null
                nodoMaior = nodoMaior.rigth
            parentNodoMaior = findParent(nodoMaior.value)
           Se nodoARemover == parentNodoMaior
                nodoARemover.left = nodoMaior.left
            Senão
                parentNodoMaior.rigth = nodoMaior.left
            nodoARemover.data = nodoMaior.data
    Devolve verdadeiro
}
```



# AVL – Remoção

```
boolean remove(Node actual, Object o) {
   nodoARemover = root
   parent = null
   pilha.add(root)
   Enquanto nodoARemover <> null e
                nodoARemover.data <> o
      parent = nodoARemover
      Se o < nodoARemover.data
         NodoARemover = nodoARemover.left
      Se não
        NodoARemover =
    nodoARemover.rigth
      pilha.add(nodoARemover)
```

```
Se nodoARemover = null
   Devolve falso
Senão
   Se root.left = null e root.rigth = null
      root = null
   Senão  root.left = null e root.rigth <> null e
 root = nodoARemover
      root = root.rigth
   Senão  root.left <> null e root.rigth = null e
 root = nodoARemover
      root = root.left
   Senão Se nodoARemover.left = null e
                nodoARemover.rigth = null
      Se nodoARemove.data < parent.data
         parent.left = null
      Senão
         parent.rigth = null
```

# AVL – Remoção

```
    Senão Se nodoARemover.left = null e nodoARemover.rigth <> null
        Se nodoARemover.data < parent.data
            parent.left = nodoARemover.rigth
        Senão
            parent.rigth = nodoARemover.rigth
    Senão Se nodoARemover.left <> null e nodoARemover.rigth = null
        Se nodoARemover.data < parent.data
            parent.left = nodoARemover.left
        Senão
            parent.rigth = nodoARemover.left
     Senão
            nodoMaior = nodoARemover.left
            Enquanto nodoMaior.rigth <> null
                nodoMaior = nodoMaior.rigth
            parentNodoMaior = findParent(nodoMaior.value)
            Se nodoARemover == parentNodoMaior
                nodoARemover.left = nodoMaior.left
            Senão
                 parentNodoMaior.rigth = nodoMaior.left
            nodoARemover.data = nodoMaior.data
        Enquanto not pilha.empty
            balance( pilha.pop() )
    Devolve verdadeiro
}
```

# Outras árvores

- Árvores Splay
- Árvores Red-Black
- Árvores AA
- Árvores B
- Árvores B+
- Árvores B*
- Etc, etc